

Artificial Intelligence

Inference in First-Order Logic

Christopher Simpkins

Kennesaw State University

1 Propositional vs. First-Order Logic

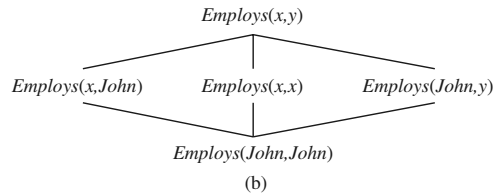
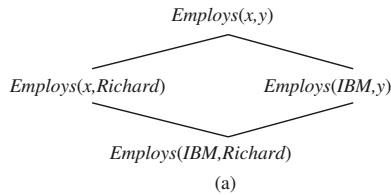
Foo

2 Unification and First-Order Inference

```
function UNIFY(x,y,θ=empty) returns a substitution to make x and y identical, or failure
if θ = failure then return failure
else if x = y then return θ
else if VARIABLE?(x) then return UNIFY-VAR(x,y,θ)
else if VARIABLE?(y) then return UNIFY-VAR(y,x,θ)
else if COMPOUND?(x) and COMPOUND?(y) then
    return UNIFY(ARGs(x), ARGs(y), UNIFY(OP(x), OP(y), θ))
else if LIST?(x) and LIST?(y) then
    return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y), θ))
else return failure
```

```
function UNIFY-VAR(var,x,θ) returns a substitution
if {var/val} ∈ θ for some val then return UNIFY(val,x,θ)
else if {x/val} ∈ θ for some val then return UNIFY(var,val,θ)
else if OCCUR-CHECK?(var,x) then return failure
else return add {var/x} to θ
```

3 Unification and First-Order Inference

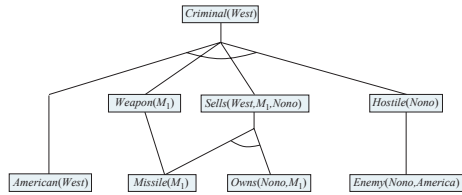


4 Forward Chaining

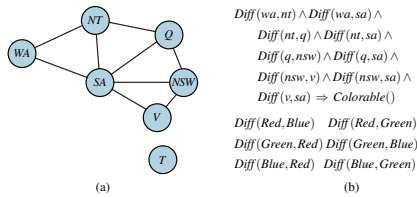
```
function FOL-FC-ASK(KB,α) returns a substitution or false
inputs: KB, the knowledge base, a set of first-order definite clauses
       α, the query, an atomic sentence

while true do
    new ← {} // The set of new sentences inferred on each iteration
    for each rule in KB do
        (p1 ∧ ... ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
        for each θ such that SUBST(θ,p1 ∧ ... ∧ pn) = SUBST(θ,p'1 ∧ ... ∧ p'n)
            for some p'1, ..., p'n in KB
                q' ← SUBST(θ,q)
                if q' does not unify with some sentence already in KB or new then
                    add q' to new
                    φ ← UNIFY(q',α)
                    if φ is not failure then return φ
    if new = {} then return false
    add new to KB
```

5 Forward Chaining



6 Forward Chaining



7 Backward Chaining

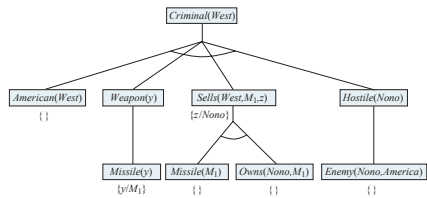
```

function FOL-BC-ASK(KB, query) returns a generator of substitutions
return FOL-BC-OR(KB, query, {})

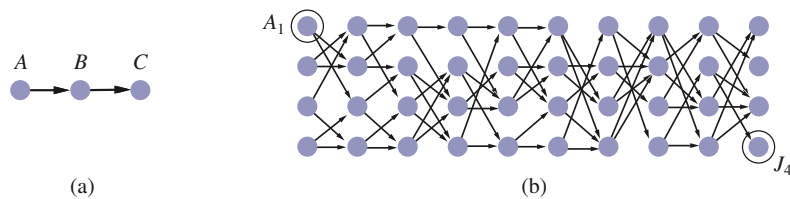
function FOL-BC-OR(KB, goal, θ) returns a substitution
for each rule r in FETCH-RULES-FOR-GOAL(KB, goal) do
    (lhs ⇒ rhs) ← STANDARDIZE-VARIABLES(rule)
    for each θ' in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
        yield θ'

function FOL-BC-AND(KB, goals, θ) returns a substitution
if θ = failure then return failure
else if LENGTH(goals) = 0 then yield θ
else
    first.rest ← FIRST(goals), REST(goals)
    for each θ' in FOL-BC-OR(KB, first, θ) do
        for each θ'' in FOL-BC-AND(KB, rest, θ') do
            yield θ''
    
```

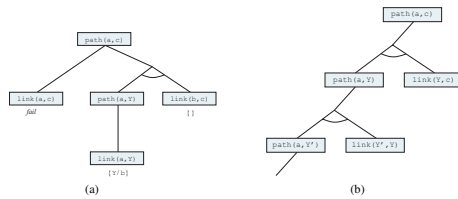
8 Backward Chaining



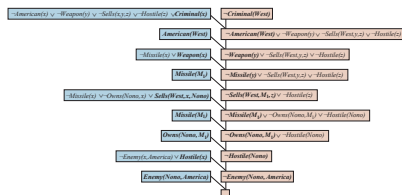
9 Logic Programming



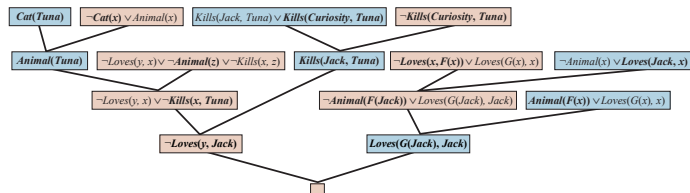
10 Logic Programming



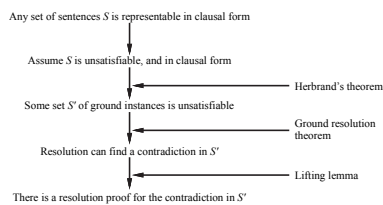
11 Resolution



12 Resolution



13 Completeness



14 G{\ "o}del's Incompleteness Theorem

Foo